Asymmetric Hierarchical Search with Dense Reward Optimization for Neural Theorem Proving

Abstract

Formal theorem proving in Lean 4 requires exact syntax, exact types, and long-horizon logical planning. This thesis proposes GammaZero, a Lean proof-search and reward-data generation framework that turns LLM samples into verified graph structure. The system separates local proof actions from skeleton actions, verifies every candidate with Lean, repairs invalid generations only for dense feedback, and performs best-first selection over open proof states using a priority queue. Skeletons decompose a proof into named subgoals that are solved in their parent theorem scaffold, with one active skeleton per parent state and reserved skeletons for fallback. Dependency-aware expression-tree analysis and AND/OR backup produce RL-compatible scored trajectories for future policy optimization. Final success always requires Lean verification of a proof with no real `sorry` or `admit` placeholder.

1 Introduction

1.1 Motivation

Large language models have shown strong ability in generating natural-language mathematical reasoning. However, a natural-language proof can hide many ambiguities: a step may omit assumptions, use an unstated lemma, rely on an informal transformation, or leave a gap that is accepted by the reader but not logically justified. Formal proof assistants address this problem by requiring every proof step to be checked in a precise logical system. Lean 4 is one such proof assistant. In Lean, mathematical statements and proofs are represented in a dependent type theory, and a theorem is accepted only when Lean can elaborate the proof and check that it has the required type. This makes Lean a useful setting for evaluating whether a generated mathematical proof is not only plausible, but formally correct.

Recent LLM-assisted proving systems therefore use large language models to propose Lean tactics, proof fragments, or complete proofs. However, the LLM is not the final judge of correctness. Lean remains the verifier: generated proof code must follow the required syntax, use available declarations, match the expected types, and close all goals. This turns theorem proving into a search problem: the LLM proposes candidate proof steps, Lean verifies them, and the search procedure decides which proof states to explore next. At the same time, strict verification also makes proof generation fragile. A small local error, such as a missing delimiter, an unavailable theorem name, or an expression with the wrong type, can invalidate the whole proof attempt.

Proving methods can be roughly grouped into three directions:

The first direction is whole-proof generation, where the model produces a complete proof in one pass. This is simple and can benefit from the model's long-form reasoning ability, but it is brittle: one local syntax, a declaration that is not available in the current context, or type error can cause the whole proof attempt to fail.

The second direction is tactic-level search. In Lean, a tactic transforms the current proof state into one or more new proof states. The model proposes local tactics, Lean executes and checks them, and the search procedure continues to explore the new proof states. Systems such as ReProver Yang et al. (2023) combine tactic generation with best-first search. This gives backtracking and partial verification, but it also creates a large search space. Many different tactic sequences may be applicable at each proof state, each tactic may require different arguments or library lemmas, and every successful tactic can generate

additional proof states that must be explored later. As proofs become longer, the search tree can grow combinatorially, requiring a large number of model samples and Lean executions before a useful proof path is found.

The third direction is proof decomposition. Instead of asking the model to solve the theorem directly or only propose the next tactic, the system asks the model to introduce useful intermediate lemmas or subgoals. This direction is motivated by the way long mathematical proofs are usually organized: a difficult theorem is reduced to smaller claims, and the final proof is assembled from those claims. Recent work such as ProD Dong et al. (2024) and Seed-Prover Chen et al. (2025) has explored lemma-based or sketch-based decomposition for formal theorem proving. Proof decomposition can reduce the difficulty of long-horizon reasoning and expose useful intermediate structure. However, decomposition also introduces new challenges. A generated subgoal may be mathematically true but irrelevant to the final proof. Large decompositions can introduce many additional subgoals, causing the search space to grow quickly. This also introduces a difficult credit assignment problem. A subgoal may be locally solvable but still provide little progress toward the original theorem. The usefulness of a decomposition is often visible only after several additional proof steps or after the final proof structure is completed.

These challenges make decomposition both promising and difficult: the system must not only generate useful intermediate claims, but also decide which decompositions are worth exploring, how to verify them, and how to connect them back to the final proof.

1.2 Proposed Approach

We propose GammaZero, a hierarchical proof-search framework for Lean 4. Instead of treating generated proofs as purely correct or incorrect, the system preserves useful local structure from partially successful proof attempts and uses it during search. The model proposes local proofs and decompositions, Lean verifies them, and the search graph records which fragments are valid, which subgoals are created, and which dependencies are needed for the final proof.

We separate proof search into two complementary modes: directly solving the current goal and introducing intermediate proof structure. This leads to two corresponding action types:

- **Local proof actions** are complete local proof attempts intended to discharge the current goal. A local proof action is accepted only if Lean reports no remaining goals for the current node.
- **Skeleton actions** are decompositions with intermediate proof structure, such as `have`, `suffices`, `cases`, or `induction`. A skeleton action may create child subgoals together with a partial proof structure describing how those subgoals are combined to solve the final theorem.

This separation turns proof search into an AND/OR graph. OR nodes represent proof states, while action nodes represent candidate proof steps. Each action forms an AND node with two types: a local proof action usually attempts to close the current goal directly and a skeleton action may create several child subgoals. A skeleton succeeds only when all of its child subgoals are solved.

To make failed generations useful for later learning, we introduce an AST-guided Sorrifier that extends placeholder-based recovery approaches such as APOLLO Ospanov et al. (2025a). The module localizes proof body failures and replaces the smallest recoverable failing region with `sorry`, producing a compilable partial script with placeholders. This patched script is not treated as a proof. It is used only to measure syntactic and structural validity.

The resulting system records how generated proofs evolve during search: which proof states are explored, which decompositions create useful subgoals, which fragments remain valid after recovery, and which intermediate claims contribute to the final proof. The model proposes; Lean verifies; the search graph records alternatives and dependencies; the Sorrifier recovers local signal from invalid code; dependency analysis measures whether skeleton subgoals matter to the final proof; and final stitching reconstructs a

proof candidate that Lean checks again. Only complete Lean verification determines final correctness, but partially valid generations can still guide search and provide intermediate reward signals.

1.3 Contributions

This thesis makes the following contributions:

- A hierarchical proof-search framework for Lean 4 that separates local proof generation from proof decomposition inside an AND/OR graph.
- A best-first search algorithm over Lean proof states using a priority queue and heuristic scoring.
- A scaffold method for solving subgoals inside the original parent proof context instead of treating them as independent theorems.
- A data synthesis pipeline that uses abstract syntax trees to recover partially valid Lean code, analyzes dependencies through Lean expression trees to evaluate the proof quality and converts proof search into structured trajectories with dense reward signals for future reinforcement learning.

1.4 Thesis Organization

Section 2 summarizes the Lean and LLM theorem-proving background needed for the proposed method. Section 3 reviews prior theorem-proving systems and benchmarks. Section 4 defines the proof state, action space, AND/OR objective, and main reward challenges. Section 6 presents the end-to-end system pipeline. Section 5 describes AST-guided recovery. Section 7 defines dense reward and credit assignment. Section 8 reserves the experimental plan. Section 9 concludes with limitations and next steps.

2 Background

2.1 Lean 4 and Formal Proof Checking

Lean 4 is an interactive theorem prover based on dependent type theory (de Moura and Ullrich, 2021). Under the propositions-as-types interpretation, a theorem is a type and a proof is a term inhabiting that type. A proof assistant verifies a theorem by elaborating user-facing syntax into typed internal terms and checking them with a trusted kernel.

This distinction between surface syntax and kernel terms is central to the design of GammaZero. The LLM generates surface-level Lean code: tactic scripts, proof blocks, local declarations, and theorem applications. Lean then elaborates that code into a lower-level expression whose type can be checked by the kernel. Many generated programs fail before they ever reach kernel checking because parsing, elaboration, type-class search, implicit argument inference, or tactic execution fails. An ATP (Automated Theorem Proving) system must therefore extract useful signal from partially successful proof attempts. A fragment may be syntactically valid but ill-typed, locally useful but incomplete, or complete only because it contains a placeholder. The framework in this thesis keeps those cases separate instead of collapsing them into a single failure label.

Lean proof scripts commonly use tactic mode:

```
theorem and_comm (P Q : Prop) :
  And P Q -> And Q P := by
  intro h
  exact And.intro h.right h.left
```

Each tactic transforms a proof state containing local hypotheses and a target goal. If all goals are discharged and the resulting proof term type-checks, Lean accepts the theorem. This kernel-checked

execution model is crucial for AI-assisted proving: models may hallucinate, but Lean remains the verifier of final correctness.

Lean also makes proof search more sensitive to context. A tactic such as `exact h` is meaningful only if `h` is in scope and has a type compatible with the current goal. A rewrite tactic depends on which simp lemmas and local equalities are available. A proof by cases or induction can change the shape of the context and create several new branches. These properties make local proof-state serialization important: the model must see enough context to generate a plausible action, and the search system must remember enough context to decide whether two states are really interchangeable.

2.2 LLM-based Theorem Proving

Recent theorem-proving systems increasingly use LLMs to generate tactics, proof terms, or whole proofs (Yang et al., 2023). Whole-proof generation is simple but fragile: one local mistake can invalidate the full output. Search-guided proving is more robust because each candidate step can be checked by Lean, and failed branches can be abandoned.

However, tactic-level search remains expensive. The model must select both tactic names and arguments, such as theorem names for `rw`, `exact`, or `apply`. In larger proofs, the key step is often not a local tactic but the discovery of an intermediate step. This motivates methods that combine neural generation with structured search and explicit decomposition.

There is also a mismatch between how mathematicians describe proofs and how a tactic-level search system explores them. A human proof often says, "It suffices to prove an auxiliary inequality," or "We first establish a monotonicity lemma". But tactic search must discover the auxiliary statement, prove it, and connect it back to the original theorem. This can make the branching factor explode. Proof skeletons are one way to utilize the high-level reasoning: the model proposes the intermediate claim explicitly, and the search system later solves the exposed subgoal.

2.3 Search-based Proof Generation

Search-guided theorem proving treats Lean as an environment. A state is the current proof state, an action is a generated proof step, and the transition is Lean execution. Common search strategies include beam search, best-first search, and Monte Carlo tree search. These methods differ in how they prioritize frontier nodes, but they share the same core difficulty: the branching factor is large and useful reward may appear only after many decisions.

Proof decomposition naturally introduces an AND/OR search structure. OR structure represents alternative proof steps that can be applied to the current proof state. AND structure appears when a decomposition generates multiple subgoals that must all be solved for the proof to succeed. Compared with a purely linear proof trajectory, an AND/OR representation is better suited for reasoning about intermediate lemmas, subgoal dependencies, and hierarchical proof structure.

2.4 Reinforcement Learning and Sparse Feedback

Reinforcement learning is a promising direction for automated theorem proving because proof generation naturally involves sequential decision making. A system repeatedly chooses proof steps, observes the resulting proof states after Lean execution, and eventually succeeds or fails depending on whether the theorem is fully verified.

However, applying reinforcement learning to Lean proof generation remains difficult. Lean provides extremely sparse feedback: a proof is either accepted or rejected, and useful signal may appear only after many proof steps. Long proofs further increase the difficulty because local actions may influence the final outcome only much later in the search process.

Dense reward signals can help guide search and future policy optimization, but they must be designed carefully. A reward based only on syntactic validity may encourage compilable but logically irrelevant proof fragments. A useful reward mechanism should therefore reflect not only local validity, but also whether generated proof fragments contribute to the final proof structure.

3 Literature Review

3.1 Neural Theorem Proving and Language Models

Neural theorem proving has increasingly moved from specialized symbolic heuristics toward systems in which neural models propose proof steps and a formal environment checks them. This separation is especially important in interactive theorem provers such as Lean, Isabelle, Coq, and HOL-style systems: the model may be uncertain, but the proof assistant provides a deterministic verification boundary. The practical question is therefore not whether the neural model can be trusted as a proof checker, but how to use a fallible proposal model to explore a very large discrete proof space efficiently.

Early LLM-based formal-proving systems often treated proof search as language modeling over tactic sequences. GPT-f demonstrated that a generative model trained on formal proof corpora can propose useful theorem-proving steps and that search around these proposals can solve nontrivial formal problems (Polu and Sutskever, 2020). The central lesson is still relevant for Lean-based systems: model samples become valuable only after they are embedded in a verification loop. A raw generated proof is fragile, but a stream of candidate steps can be filtered, ranked, and combined by an external search procedure.

This thesis follows the same verifier-centered philosophy but changes the unit of search. Instead of treating every generated step as a flat continuation, GammaZero distinguishes local proof actions from skeleton actions. A local proof action is an attempt to finish the current local state; a skeleton is a proposed decomposition that creates named subgoals. This makes the search graph closer to the mathematical structure of a proof, where an intermediate lemma is useful only if it is eventually proved and consumed by the final argument. The contribution is therefore not a new foundation for formal verification, but a search-and-credit structure that makes LLM proposals easier to verify, repair, score, and reuse.

3.2 Search Procedures for Formal Proofs

Search is unavoidable because formal theorem proving has sparse terminal feedback. A proof assistant usually provides a binary signal for a complete candidate: the proof either elaborates and type-checks, or it does not. Search procedures create intermediate decision points so that failed branches can be abandoned without discarding all progress. Beam search, best-first search, Monte Carlo tree search, and tree-structured proof search all instantiate this idea, but they differ in how they allocate effort across open states.

HyperTree Proof Search studies proof search as a structured exploration problem rather than a single linear sequence (Lample et al., 2022). This is conceptually close to the AND/OR perspective used in GammaZero. When a proof step creates several subgoals, solving only one branch is insufficient: the decomposition succeeds only if every required child is solved. This hard conjunction is easy to lose in linear trajectory formulations, where partial progress on a subgoal may look better than it should. The AND/OR view makes that dependency explicit.

GammaZero differs from a pure tree-search formulation in two ways. First, it uses Lean scaffolds to keep subgoals attached to the exact parent proof in which they were created. A child goal is not simply a standalone theorem with a similar statement; it is a placeholder position inside a parent proof body. Second, GammaZero uses dense reward information from failed or partially repaired candidates. This means search is not only a solver but also a data-generation process: even branches that do not solve the root theorem may produce useful supervision about syntax survival, dependency quality, and action type.

3.3 Retrieval, Premise Selection, and Hammers

Many successful theorem-proving systems rely on retrieval or premise selection. In large mathematical libraries, the hard part is often not inventing a new proof idea but finding the relevant theorem, lemma, definition, or rewrite rule. LeanDojo introduced an environment for interacting with Lean and studied retrieval-augmented theorem proving through ReProver (Yang et al., 2023). Its results emphasize that LLM proof search benefits from access to library context rather than relying only on the local goal.

Hammer-style systems pursue a related goal by connecting interactive proof assistants to automated theorem provers and premise-selection machinery. Thor integrates language models with external automated theorem provers, showing that neural guidance and symbolic automation can complement each other (Jiang et al., 2022a). The language model can propose or guide a search, while automated tools can discharge subgoals once relevant premises have been identified. This pattern is important because it suggests that LLMs need not solve every local goal directly; they can also create structure that makes other solvers more effective.

GammaZero is compatible with retrieval and hammer-style extensions, but it does not depend on them in the current thesis. Its contribution lies one layer lower: it defines how generated actions are verified, classified, repaired, inserted into a proof graph, and assigned credit. A future version could augment local proof prompts with retrieved premises or allow skeleton prompts to request relevant lemmas, but the current architecture first establishes the correctness and reward boundaries for hierarchical search.

3.4 Sketching, Decomposition, and Informal Guidance

Several recent systems use informal reasoning or high-level sketches to guide formal proof search. Draft, Sketch, and Prove uses informal proof sketches to guide formal theorem provers, reflecting the observation that high-level mathematical plans are often easier to generate than fully elaborated formal proofs (Jiang et al., 2022b). This line of work is closely related to GammaZero's skeleton actions: both approaches separate the discovery of proof structure from the final discharge of low-level subgoals.

The difference is that GammaZero makes the sketch executable as a Lean scaffold. A skeleton is not just a natural-language plan. It is a Lean proof body with named placeholders, such as local declarations introduced by `have`. This gives the system a precise interface: Lean can elaborate the partial structure, report the child proof states associated with placeholders, and later verify the final stitched proof. The cost of this precision is that skeletons must obey stricter syntactic and semantic constraints than informal sketches. The benefit is that every decomposition is grounded in the same formal context as the final proof.

This distinction motivates scaffold-aware subgoal solving. If a skeleton introduces several subgoals, the system should not solve each child as if it were an independent theorem detached from its parent proof. Local declarations, binder structure, and sibling placeholders all affect what counts as a valid replacement. GammaZero therefore focuses one target `sorry` while sibling subgoals are temporarily replaced by `admit`; the model is asked to solve exactly the focused placeholder and preserve the rest of the scaffold. This design turns high-level decomposition into a verifiable editing task.

3.5 Benchmarks and Evaluation Datasets

The miniF2F benchmark has become a standard evaluation set for formal olympiad-style mathematics (Zheng et al., 2022). It is useful because it contains problems whose statements are formalized in proof assistants, enabling objective measurement of proof success. However, the benchmark also illustrates a broader challenge: formal benchmark statements may diverge from the intended informal problem, and small differences in theorem statements can substantially change difficulty. The miniF2F-Lean revisited work highlights these issues and motivates careful dataset selection and reporting (Ospanov et al., 2025b).

ProofNet broadens the evaluation perspective by targeting undergraduate-level mathematics and autoformalization (Azerbayev et al., 2023). It is relevant here because a hierarchical prover must eventually operate across theorem statements that vary in style, domain, and amount of implicit context. Problems from algebra, number theory, inequalities, and elementary analysis often require different kinds of intermediate claims. A skeleton mechanism should therefore be evaluated not only by whether it solves more roots, but also by whether it proposes subgoals that are meaningful, solvable, and used.

This thesis evaluates GammaZero primarily as a search and reward-data generation framework. The reported metrics are therefore broader than root solve rate alone. Root solve rate remains the strongest correctness metric, but the system also records generated actions, verification calls, repair rates, skeleton commitment statistics, dependency classes, and queue pruning counts. These measurements are needed because the framework is designed to generate training data for future optimization, not merely to report a single pass rate.

3.6 Autoformalization

Autoformalization studies the translation of informal mathematical statements, proofs, or problem descriptions into a formal language that can be checked by a proof assistant. This problem is adjacent to formal theorem proving but not identical to it. A prover starts from a formal statement and attempts to construct a proof term or tactic script. An autoformalizer may instead start from natural language and must decide which definitions, coercions, quantifier order, side conditions, and library notions capture the intended mathematics. A formally verified proof of the wrong statement is still a verification artifact, but it does not solve the original informal problem.

Large language models have made autoformalization more plausible because they can map between informal mathematical prose and formal syntax when given sufficient examples. Wu et al. study autoformalization with large language models and show that translation quality can improve when informal and formal corpora are used together (Wu et al., 2022). The central difficulty is semantic alignment: the model must preserve the meaning of the informal statement while selecting a formal encoding accepted by the target proof assistant. This is especially delicate in mathematics because many informal statements suppress assumptions that a formal theorem must make explicit.

ProofNet highlights this distinction by pairing undergraduate-level mathematical problems with formal counterparts and by evaluating both autoformalization and proving (Azerbayev et al., 2023). Such datasets are valuable because they expose two separate failure modes. A system may fail before proof search begins because the formal statement is missing a hypothesis or uses an unsuitable definition. It may also receive a correct formal statement but fail to prove it. The miniF2F-Lean revisited analysis similarly motivates caution when interpreting formal benchmark results: the exact formal statement and library context can alter the problem being measured (Ospanov et al., 2025b).

GammaZero does not attempt autoformalization in this thesis. The input is assumed to be an existing Lean theorem statement, and the system's responsibility begins after that statement is fixed. This boundary is important for evaluation. A failed GammaZero run should be interpreted as a failure to find a verified proof within the current search limits, not as evidence that the informal theorem is false or that the formalization is faithful. Conversely, a successful run certifies the formal theorem that Lean checked; it does not by itself validate an upstream natural-language translation. Future systems could connect an autoformalizer to GammaZero, but the correctness boundary would still need to distinguish translation validity from proof validity.

3.7 Reinforcement Learning for Theorem Proving

Reinforcement learning is attractive for theorem proving because proof construction is naturally sequential. A prover chooses an action, observes a new state from the proof assistant, and eventually receives a success or failure signal. Earlier work on reinforcement learning of theorem proving explored how learned

guidance can improve symbolic proof search and how proof attempts can produce training signal for subsequent attempts (Kaliszyk et al., 2018). HOList made this direction more systematic by providing an environment for machine learning over Higher Order Logic theorem proving, with explicit states, actions, and goals suitable for learned policies (Bansal et al., 2019).

The main obstacle is reward sparsity. A theorem prover may need many local decisions before the final checker accepts the proof. Most intermediate states do not come with a natural scalar value, and failed branches may still contain useful mathematical structure. This makes a naive terminal reward difficult to use: the system learns only that a complete trajectory failed, without knowing whether the early plan was bad, the final tactic was malformed, or a promising subgoal was simply not finished in time. Proof-search systems therefore often combine policy guidance with search, replay, or additional value estimates rather than relying on single sampled trajectories.

Recent neural provers continue this pattern. GPT-f uses a generative model and proof search rather than trusting one direct completion (Polu and Sutskever, 2020). DeepSeek-Prover-V1.5 explicitly combines proof assistant feedback with reinforcement learning and Monte Carlo tree search, showing the importance of using checker feedback as a learning signal rather than treating the language model as a standalone reasoner (Xin et al., 2024). These systems support a broader lesson: reinforcement learning for formal proving is most credible when it is grounded in verified transitions from a proof assistant.

This thesis does not train a reinforcement learning policy. Instead, GammaZero is designed to generate RL-compatible scored trajectories. Each verified, repaired, failed, or partially useful action can be recorded with environment feedback, dependency information, graph position, and final proof outcome. The goal is to make future optimization more data-rich: a policy should be able to learn not only which actions solved a root theorem, but also which skeletons introduced useful child proof states, which local tactics survived Lean checking, and which declarations were unused in the final proof.

3.8 Positioning of GammaZero

Existing work suggests several complementary directions for LLM-assisted theorem proving. Search-based systems treat Lean as a verification environment and explore many candidate proof steps. Sketch-based systems introduce higher-level proof structure through intermediate lemmas or informal plans. Retrieval-based systems improve local reasoning by exposing relevant library context. Autoformalization systems address the upstream problem of producing faithful formal statements from informal mathematics. Reinforcement learning systems study how proof-assistant feedback can improve future proposal policies.

GammaZero is positioned between flat tactic search and high-level proof sketching. Instead of generating one tactic at a time, the framework separates local proof generation from explicit proof decomposition. Local proof actions attempt to close the current proof state directly, while skeleton actions introduce intermediate structure through executable Lean scaffolds with named subgoals.

This separation changes the role of search. The search graph does not expand after every tactic step. New proof states are introduced mainly through decomposition decisions, which makes the growth of the search space depend more on skeleton structure than on low-level tactic exploration. At the same time, every decomposition remains grounded in Lean through elaborated scaffolds and verified child proof states.

GammaZero also differs from purely success-driven proof search. Failed or partially valid generations are not discarded immediately. Instead, the framework extracts repair information, dependency structure, and local verification signals to produce structured search trajectories for future reinforcement learning and policy optimization. In that sense, GammaZero occupies a deliberately narrow but useful position: it assumes the theorem is already formalized, keeps Lean as the correctness authority, uses best-first search to spend effort on promising open states, and records dense feedback that can support future policy learning.

4 Problem Formulation

4.1 Proof States and Proof Scaffolds

Given a theorem statement T, the objective is to construct Lean code that proves T without using `sorry`, `admit`, or any temporary placeholder. Lean verification is the final correctness criterion: a proof is accepted only when the complete theorem is checked successfully.

GammaZero represents proof search as a graph of open proof states. A proof state is not only a goal formula. It also includes the proof context in which that goal appears. This distinction is important because the same goal may occur in different parts of a proof and may have access to different local facts.

To preserve this context, GammaZero uses a proof scaffold. A scaffold is a partial Lean proof that contains the current hole to be solved, the surrounding proof code, and the structure that connects this hole back to the parent goal. The model is therefore not asked to solve a child goal as an independent theorem. Instead, it solves the goal inside the proof structure where the goal was created.

Each open state is assigned one target hole. The task of the state is to replace that hole with valid Lean code. Other unfinished holes in the same scaffold are treated as sibling holes. They are not solved by the current state; they belong to other states in the graph. This gives each state a clear local task while still keeping the full parent proof structure visible.

This design is especially important for skeleton actions. A skeleton action may introduce several child subgoals, but it also provides a partial proof structure showing how those subgoals are intended to support the parent goal. Solving the children separately is not enough by itself. Their proofs must later fit back into the scaffold so that the whole parent proof can be checked by Lean.

At a high level, a scaffolded proof state can be viewed as

$$s = (C, i),$$

where C is the proof scaffold and i identifies the target hole inside that scaffold. Lean provides the local context and the target goal at this hole, and these are shown to the model to make the task explicit. However, the state is not represented only by the pair of local context and goal. The surrounding scaffold is part of the state because it records where the hole occurs and how the completed proof will connect back to the parent proof. This distinction matters because two holes may have the same visible goal but occur in different scaffolds. A proof that is valid for one hole is not necessarily valid for the other. This gives each state a simple contract: it may replace the target hole with Lean code, but it must leave the surrounding scaffold unchanged.

After the required child states are solved, GammaZero inserts their proof bodies back into the parent scaffold and asks Lean to verify the resulting proof again. Temporary placeholders used during search do not count as proof. They are only a way to check one part of a larger proof while the other parts are still being solved. Final success always requires complete Lean verification without placeholders.

4.2 Action Space: Local Proof Actions and Skeleton Actions

GammaZero separates two ways of continuing proof search. The first way is to solve the current proof state directly. The second way is to introduce proof structure that may create smaller subgoals. This gives two action types:

$$\mathcal{A} = \mathcal{A}_{local} \cup \mathcal{A}_{skel}.$$

Local proof actions. A local proof action is a generated Lean proof body for the current target hole. Its goal is to close the current proof state inside the current scaffold. The action may contain multiple tactics and may temporarily create subgoals during execution. However, it is accepted as a successful local proof action only if all of these goals are solved by the end of the generated proof body.

If a local proof action leaves any unresolved goal, it fails as a local proof action. It is not converted into a decomposition and it does not create child proof states in the search graph. This rule keeps the meaning of local proof actions simple: they either solve the current state or fail at that state.

This design reduces the number of search decisions inside one proof state. In a tactic-level search, each small tactic step may become a new search decision. In GammaZero, a local proof action represents a complete local attempt. Since it does not expand the graph, the growth of the graph is controlled mainly by skeleton actions.

Skeleton actions. A skeleton action is a generated Lean proof body that introduces intermediate proof structure. It may use constructs such as `have`, `suffices`, `cases`, or `induction`. Unlike a local proof action, a skeleton action is allowed to create child subgoals.

A skeleton action does not merely list subgoals. It also describes how the resulting proof pieces are intended to support the current goal. For example:

```
intro hp
have hq : Q := by
  sorry
exact h2 hq
```

The intermediate statement Q becomes a child subgoal, while the last line shows how the completed proof of Q will be used to continue the parent proof.

A skeleton action may create zero or more child subgoals. If it creates no child subgoal and the current state is solved, it is simply a successful action. If it creates child subgoals, then all of them must eventually be solved for the skeleton to succeed. In the search graph, the child subgoals become child proof states.

This gives the graph an AND/OR structure. OR nodes are proof states, because a proof state can be solved by any successful outgoing action. Skeleton actions have AND semantics, because every child proof state created by the skeleton must be solved. This structure matches proof decomposition: a decomposition is useful only when all required subgoals can be completed and connected back to the parent proof.

The concrete running example for these definitions is moved to Appendix A.

4.3 AND/OR Graph Semantics

The action definitions above induce an AND/OR graph over scaffolded proof states. OR nodes are proof states: a state is solved when at least one outgoing action succeeds. Action nodes represent generated proof attempts. A local proof action is an action with no child proof state after success. A skeleton action may have child proof states, and it succeeds only when all required children are solved and the completed scaffold is accepted by Lean.

The graph makes two different relations explicit. A proof state represents a choice among candidate actions: any successful action can solve the state. A skeleton represents a decomposition: every child proof state created by the skeleton must be solved before the skeleton can solve its parent state.

GammaZero uses a graph rather than a tree because duplicate states can be shared. However, state identity is scaffold-sensitive. Two states are merged only when they refer to the same target problem inside the same proof scaffold. Thus, a child subgoal is not merged back into an ancestor merely because the visible goal text is the same. The scaffold records where the goal occurs and how it connects to the parent proof, so equal-looking goals in different scaffolds remain distinct.

Solved status propagates through this graph. When a child proof state is solved, its parent skeleton may become solved. When a skeleton action is solved, its parent proof state may become solved. This propagation can continue upward until the root theorem is solved.

This graph view is also used later for reward backup. OR nodes select among alternative actions, while skeleton actions are limited by their unfinished children. The full reward formulas are defined in Section 7; here the important point is only the logical dependency: alternatives are OR choices, and decomposition children are AND requirements.

4.4 Search Limits and Failure Labels

GammaZero runs proof search under finite limits. These limits include the number of generated actions, the maximum search depth, the number of local proof attempts and skeleton attempts per state, the number of open states kept in the queue, model generation limits, and Lean verification timeouts. These limits are necessary because both model sampling and Lean verification are expensive.

Because the search is finite, the labels used in the graph are operational labels. They describe what happened during the current run; they do not describe mathematical truth. In particular, failing to solve a state within the current limits does not mean that the goal is false or impossible to prove.

The main status labels are:

Status	Meaning
SOLVED	The state or action has been closed by Lean without remaining placeholders.
OPEN	The state is still active and may be expanded later.
FAILED	The action, state, or skeleton cannot continue successfully in the current run.
UNRESOLVED WITHIN LIMITS	The search stopped before the open state was solved.

A local proof action is marked successful only if Lean verifies that the current target has been fully solved. If it leaves an unresolved goal, it fails as a local proof action. The failed action may still be kept in the record for reward computation or analysis, but it does not create child proof states and does not solve the parent state.

A skeleton action has a stricter success condition. If it creates child proof states, then every child must be solved and the completed scaffold must be accepted by Lean. If one required child remains unsolved when the search gives up on that decomposition, the skeleton is marked failed in the graph. This does not mean that the child goal is mathematically impossible. It only means that the skeleton was not completed under the current search limits.

A proof state is solved when at least one outgoing action succeeds. A proof state may remain open while there are still allowed attempts or while one of its active decompositions is still being explored. If all useful attempts have failed or the state has exhausted its allowed search budget, it may be marked failed for the current run. If the global search stops while the state is still open, it is recorded as unresolved within limits.

This distinction is important for reward assignment. A repaired candidate may provide useful reward signal, but it cannot solve a state if placeholders remain. A skeleton with one solved child and one unresolved child may contain useful structure, but it is not a solved decomposition. The system may keep immediate rewards and diagnostic information from such actions, while withholding delayed structural credit until the required proof states are actually solved.

Timeouts and system failures are treated in the same operational way. They are recorded as failed actions or interrupted checks for the current run, not as evidence about the truth of the theorem. The final correctness boundary remains Lean verification of the completed proof: the root theorem is solved only when Lean accepts the fully assembled proof without `sorry`, `admit`, or any temporary placeholder.

4.5 Formal Invariants

The formulation is governed by a small set of invariants. First, every scaffolded proof state has exactly one target hole. This target is the only location that the current state is responsible for solving. Other unfinished holes in the same scaffold belong to sibling states.

Second, while one target is being solved, sibling subgoals may be represented by temporary `admit` placeholders. These placeholders are part of the surrounding scaffold and must not be changed by the candidate action. The current action may replace only the target hole.

Third, a local proof action can solve the current target, but it cannot create child proof states. It may use multiple tactics internally, and those tactics may create temporary goals during execution. However, all such goals must be closed by the end of the generated proof body. If any goal remains, the action fails as a local proof action and does not expand the graph.

Fourth, a skeleton action may create child proof states by extending the current scaffold with named child subgoals. If it creates child states, the skeleton cannot solve its parent until every required child has been solved and the completed scaffold passes Lean verification. If a skeleton creates no child state, it is solved only when Lean verifies that it directly closes the current target.

Fifth, a proof state can be marked SOLVED only through Lean verification. For a local state, the replacement for the target hole must not contain `sorry`, `admit`, or any temporary placeholder. For a parent skeleton, all child proof bodies must be stitched back into the parent scaffold and the completed block must be checked again. A repaired script that still contains placeholders may provide reward information, but it cannot solve a state.

The global invariant is that temporary placeholders are never part of an accepted theorem. They are used only to isolate responsibilities during search. Once the graph claims that a parent skeleton is solved, GammaZero inserts the solved child proof bodies into the parent scaffold and asks Lean to verify the assembled proof. The root theorem is accepted only after this recursive assembly reaches the root and Lean verifies the complete theorem without unresolved placeholders.

These invariants also clarify how failure is represented. Parser errors, elaboration errors, timeouts, and policy-check rejections are failed actions, not failed theorems. An open state that is not solved within the current search limits remains open from the mathematical point of view, even if the exported record labels it as UNRESOLVED WITHIN LIMITS. This distinction is important for future training: the model should learn that the explored branch did not succeed under the current search configuration, but it should not learn that the underlying goal is false.

5 AST-Guided Sorrifier

5.1 Motivation

Lean failures are often local. A generated proof may contain a valid prefix and one broken command, but normal binary verification rejects the whole script. The Sorrifier recovers partial signal by replacing localized failing regions with `sorry`.

We distinguish two notions:

- **Placeholder-compilable script:** Lean accepts the script only because unresolved or invalid regions were replaced with `sorry`.
- **Fully verified proof:** Lean accepts the script with no `sorry` placeholder.

The Sorrifier is used only for reward estimation and partial-credit extraction. A theorem is considered solved only when Lean accepts the proof without any `sorry`.

5.2 Failure Taxonomy

Generated Lean failures are categorized conceptually as follows:

- **Parser errors:** malformed syntax, such as missing brackets, invalid tactic syntax, or broken proof blocks.
- **Elaboration errors:** unresolved names, missing implicit arguments, or expressions that Lean cannot elaborate.
- **Type errors:** syntactically valid terms whose inferred types do not match the expected goal.
- **Unsolved goals:** valid proof fragments that execute but leave remaining subgoals.

This taxonomy is used to explain the main failure modes observed in generated Lean code. The current implementation does not rely on a complete classifier that formally separates parser, elaboration, and type-checking failures. Instead, candidate actions are executed through a persistent Lean worker, while an AST daemon provides syntax spans when a parse tree is available. The worker returns a structured result containing diagnostics and proof-state metadata. The main returned fields are `errors`, `warnings`, `infos`, and `sorries`. Each diagnostic contains a severity level, a textual message, and a source position:

```
{
  "pos": {"line": ..., "column": ...},
  "endPos": {"line": ..., "column": ...},
  "severity": "error" | "warning" | "info",
  "data": "..."
}
```

Operationally, the implementation performs only a coarse separation between unresolved-goal cases and fatal failures. Diagnostics indicating remaining goals are handled together with the `sorries` field, which provides the proof states associated with placeholder locations. Other diagnostic failures are passed to the Sorrifier as recoverable fatal errors. Worker crashes, system failures, and timeouts are not repair targets; they are logged as failed actions.

The recovery pipeline is therefore diagnostic-driven rather than taxonomy-driven. It starts from the first Lean error location, or from the first unsolved-goal location when the candidate elaborates but does not close the expected target. Given this source position, the Sorrifier attempts to identify the affected region and replace it with a placeholder. If an AST can be constructed, the system selects the smallest enclosing tactic, tactic sequence, expression, or declaration block that covers the reported source position. If the AST is only partial or cannot be constructed due to severe syntax failure, the system falls back to a coarser line-, indentation-, or block-based recovery strategy. This fallback searches for nearby structural boundaries such as `have`, bullet blocks, or `:= by` proof bodies.

Different patch units are used depending on how precisely the failure can be localized. Local errors may be patched at the level of a single tactic line or AST node. Repeated failures or severe malformed blocks may trigger a larger structural patch that removes or replaces an enclosing command block. If the failure occurs in a declaration body and local recovery is unreliable, the system may replace the body with a placeholder proof:

```
:= by
  sorry
```

Unsolved goals are handled differently from fatal failures. For skeleton actions, unresolved goals reported through `sorries` are converted into child OR nodes in the AND/OR graph. For local proof actions, remaining goals mean that the action did not discharge the current node. Such actions may still receive immediate recovery reward, but they are not allowed to expand the graph. A more fine-grained classifier for parser, elaboration, and type errors is left as a possible implementation refinement.

5.3 AST-based Error Localization

Let π be a generated script and $\mathcal{T}(\pi)$ its AST. When Lean reports a source span, the Sorrifier maps that span to the narrowest enclosing syntax node. The preferred repair units are a tactic, a tactic sequence, an expression, or a declaration proof body. The system patches the smallest viable unit first, preserving the largest possible amount of surrounding generated code.

If the error belongs to a declaration body, the body can be replaced by a placeholder proof:

```
:= by
  sorry
```

If the error belongs to a local tactic or expression, only that local region is replaced where possible.

5.4 Iterative Patching Algorithm

The recovery loop is:

1. Submit the generated script to Lean.
2. If it verifies without placeholders, return full success.
3. If Lean reports an error span or unsolved-goal span, locate the smallest enclosing AST node.
4. Replace the failing node with `sorry` or a placeholder proof block.
5. Re-submit the patched script.
6. Stop when the script becomes placeholder-compilable or the patch limit is exhausted.

To avoid oscillation, the implementation stores hashes of intermediate patched scripts. If a repeated broken state is detected, it escalates to a larger enclosing structural block. If no reliable local repair remains, the action is recorded as failed for search while retaining diagnostics for analysis.

The iterative nature of the algorithm matters because Lean errors often cascade. A single missing argument can make later lines type-check in a different local context, and an unknown identifier can cause subsequent tactics to report misleading errors. Patching only the first error is therefore not always enough. GammaZero repeatedly submits the patched candidate to Lean so that the next diagnostic is computed in the updated proof state. This makes the repair process conservative: each patch is justified by a concrete Lean response, rather than by a purely textual guess about which lines look suspicious.

At the same time, the repair loop is not allowed to become a proof search procedure of its own. It does not invent new theorem names, synthesize missing arguments, or replace a broken line with a semantically different tactic. Its role is to preserve the largest checkable prefix or surrounding structure by inserting placeholders where the generated code fails. This keeps the reward interpretation simple. A high survival score means that much of the model's original code remained Lean-checkable after local placeholder insertion; it does not mean that the system found a new proof by repairing the code.

5.5 Dense Syntactic Reward Extraction

After recovery, the system estimates how much of the original generated proof body survives in the patched, placeholder-compilable script. Although AST information may be used to localize failing regions during recovery, the current reward implementation does not count AST nodes directly. Instead, it uses a line-based survival score over cleaned proof lines.

The proof body is normalized by removing blank lines and comment-only lines. The remaining lines from the original generation are compared with the lines in the patched script using sequence matching. Lines that remain unchanged and in the same relative order are counted as surviving. Lines removed by patching, deletion, or orphan-code cleanup are treated as non-surviving.

This produces an immediate recoverability signal: generations that require only small localized patches receive higher reward, while generations whose proof bodies must be mostly replaced by `sorry` receive lower reward. The exact formula, including the penalty for unused or ineffective code diagnostics, is defined in Section 7.

The line-based score is deliberately coarse. Counting AST nodes would make the reward sensitive to parser details and to how Lean represents syntactic sugar. Counting surviving lines instead aligns the signal with the model output that is visible in the trajectory record. A short proof whose only line fails should receive little immediate credit; a longer skeleton that contains several valid declarations before a localized failure should receive more. This is not a proof-quality metric, but it is useful feedback for future optimization because it distinguishes completely unusable generations from generations that are nearly well formed.

There are also cases where recovery is intentionally disabled. Timeouts, worker crashes, and system-level failures do not identify a trustworthy local source span, so replacing text with `sorry` would create a misleading reward signal. Similarly, action-policy violations such as using `admit` in a skeleton are not treated as ordinary Lean failures. They indicate that the model did not follow the action contract and are recorded as rejected candidates.

A limitation of line-based survival is that it does not measure semantic importance. A single surviving line might introduce a crucial intermediate lemma, while several surviving lines might only set up unused local names. This is why r_{env} is paired with dependency reward rather than used alone. The immediate reward answers a narrow question: how much generated Lean structure survived recovery? The dependency reward answers a later question: did that structure matter after the proof was stitched and elaborated? Keeping these rewards separate makes the trajectory more interpretable.

Another limitation is that repair can make bad model behavior look superficially organized. A model that repeatedly emits long invalid scripts may still obtain nonzero survival rewards if some prefixes parse. For this reason, failed actions do not solve states and repaired skeletons receive a mild priority penalty. The search can learn from repaired code, but it still prefers clean actions when other signals are similar.

Concrete Sorrifier examples and line-based reward calculations are moved to Appendix B.

6 Proposed Framework

6.1 System Components

GammaZero is organized around four interacting components:

- **Model interface:** asks the LLM for local proof actions or skeleton actions under separate output contracts.
- **Lean verification interface:** checks generated candidates, reports diagnostics, returns placeholder states, and rejects candidates that violate the action contract.
- **Sorrifier:** repairs invalid candidates with `sorry` only for partial reward estimation.
- **Search and reward evaluator:** maintains the AND/OR graph, selects open states using a priority queue, computes rewards, and exports scored trajectories.

These components implement the formulation from Section 4. The model proposes candidate proof code, Lean checks it, the graph records the resulting state changes, and the reward evaluator stores both successful and failed attempts for later analysis.

6.2 End-to-End Lifecycle

A typical GammaZero run begins with a theorem whose proof body contains one root placeholder. Lean is asked for the initial proof state, and the state is inserted into the priority queue. The search loop then selects an open state, serializes the scaffold, target goal, and local context, and asks the model for local proof attempts. If a local proof action closes the state, the state becomes solved and the solution can propagate upward. If local proof attempts fail but still receive high recoverability scores, the state may receive more local proof attempts. If local closure appears unlikely, the state becomes eligible for skeleton generation.

When the model proposes a skeleton, Lean checks whether the parent proof body can elaborate with named placeholders. The system extracts each placeholder as a child proof state and constructs a scaffold for solving that child. The best skeleton for the parent is committed, and its child states are inserted into the priority queue. Other promising skeletons may be reserved. Search then continues from the highest-priority open child, which may itself be solved by a local proof action or decomposed by another skeleton.

When a child is solved, its proof body is inserted into the corresponding placeholder of its parent skeleton. If all children of a committed skeleton are solved, the skeleton becomes a solved action and the parent state becomes solved. This propagation can continue until the root theorem is solved. The final theorem is then reconstructed from the selected solved actions, cleaned through dependency analysis where possible, and verified again by Lean. The exported graph record still contains failed actions, repaired candidates, reserved skeletons, and pruned states, but the final proof contains only the selected verified path.

6.3 Scaffold Contract in the Implementation

The scaffold contract from Section 4 is enforced at the level of candidate extraction. When a skeleton creates child subgoals, each child is checked inside a scaffold derived from the parent proof. The focused child is represented by the target `sorry`, while sibling children are represented by temporary `admit` placeholders.

The model may see the surrounding scaffold, but the accepted action is only the replacement for the target hole. If the returned code changes a sibling `admit`, removes surrounding declarations, or modifies the parent assembly, the action is rejected or recorded as an extraction failure. The local contract is:

> solve exactly this `sorry`; keep every `admit` unchanged.

Each state with a scaffold records the scaffold hash, target kind, target index, and, when available, the target label inferred from the declaration that contains the target. If the target placeholder appears in `have h_step : P := by sorry`, the target label is `h_step`. These fields are part of state identity, so two equal-looking goals are not conflated when they occupy different scaffold positions.

This representation is also used for nested skeletons. A local proof action must replace only the target placeholder and must not modify sibling `admit` placeholders. A child skeleton may introduce further named child subgoals, but the extracted replacement is still checked against the original scaffold before it is accepted.

Target labels and target indices make this check stable. A label helps humans read the log; the index helps the system identify the exact placeholder when several similar declarations appear in a single proof body. Together they make the extracted action robust to superficial formatting changes.

6.4 Model Interface and Output Parsing

The model is used as a fixed proposal mechanism. For local proof requests, the prompt asks for Lean proof code that closes the current target and forbids `sorry` and `admit`. For skeleton requests, the prompt

asks for a proof outline with named child subgoals. `admit` is forbidden, and `sorry` is allowed only for named child subgoals, for example:

```
have h_step : P := by
  sorry
```

The prompt may include optional natural-language reasoning, but only the Lean code block is passed to Lean. The parser extracts the first valid Lean code block, stops at the configured generation limit, and applies action-specific checks before verification. For subgoal scaffolds, the parser compares the returned full scaffold with the expected scaffold and extracts only the replacement for the target `sorry`.

6.5 Lean Verification Interface

Candidates are executed by persistent Lean workers. The returned result includes diagnostics, warnings, placeholder locations, generated proof states, and enough source-position metadata for repair and subgoal extraction. A local proof action succeeds only if Lean closes the current state with no real `sorry` or `admit`. A skeleton is accepted as a candidate when it passes the skeleton format checks and either closes the current target directly or exposes valid child proof states.

Failed candidates may be sent to the Sorrifier for immediate reward estimation. Repair does not turn a failed local proof action into a solved state, and it does not turn a placeholder-containing skeleton into a final proof. System failures, worker crashes, and timeouts are recorded as failed actions with metadata, not repaired as proof fragments.

6.6 Priority-Queue Search

GammaZero performs best-first selection using a priority queue over open proof states. A run has explicit resource limits, including a maximum number of generated actions, maximum depth, maximum local proof attempts per state, maximum skeleton attempts per state, a global open-state limit, per-depth open-state limits, Lean timeouts, and model generation limits.

Algorithm 1: Priority-Queue Hierarchical Search

1. Initialize the root OR node from the theorem scaffold and insert it into the priority queue.
2. Pop a high-priority open state.
3. Try local proof candidates first, subject to the per-state and global action limits.
4. Verify or repair candidates, record rewards, and mark the state solved if a local proof action closes it without placeholders.
5. If local proof attempts are insufficient or unsuccessful, decide whether the state should request skeletons.
6. Generate, verify, repair, and score skeleton candidates.
7. Commit at most one active skeleton for the parent state; store other promising skeletons as reserved skeletons.
8. Insert child states of the active skeleton into the priority queue.
9. Prune low-priority open states when global or per-depth queue limits are exceeded.
10. Stop when the root is solved or the generated-action, depth, open-state, timeout, or verification-call limits are reached.
11. Run dependency analysis, final stitching, and bottom-up AND/OR reward backup.

Micro-batching is used only as an execution schedule for model calls and Lean verification. It does not define the logical order of expansion. The logical search order is determined by priority scores over open states.

6.7 Local-Proof-First Expansion

The search does not request skeletons immediately for every state. For a new state, GammaZero first samples local proof actions because many Lean goals can be closed inside the current scaffold and should not be decomposed. A skeleton request is made only after local proof attempts fail to solve the state, after the state has used enough local attempts, or after priority signals indicate that decomposition may be useful.

The rule is adaptive. If failed local proof attempts have high r_{env}, the system may continue local proof sampling longer because the model is producing Lean code that mostly survives verification and repair. If local proof attempts have low recoverability or repeatedly leave the same state open, the state becomes eligible for skeleton generation earlier.

6.8 Skeleton Commitment and Fallback

Several skeletons can be generated for the same proof state, but they may organize the proof around different intermediate structures. Solving children from one skeleton does not necessarily help complete another skeleton. GammaZero therefore commits to one active skeleton per parent state, while keeping other promising skeletons as reserves. The committed skeleton receives focused search effort: its children enter the priority queue, and the parent does not keep requesting unrelated skeletons while that skeleton remains active.

Other promising skeletons are stored as reserved skeletons. If the committed skeleton fails, reaches its search limits without progress, or becomes stale after repeated rounds, a reserved skeleton can be activated. Duplicate skeletons are rejected before reservation when they expose the same scaffold structure and equivalent child subgoals. This keeps search effort concentrated without making the first skeleton choice irreversible.

6.9 Priority Scores

The priority scores are heuristics for allocating limited search effort. They are not intended to measure mathematical truth or proof elegance. Their role is to decide which open state should receive the next model and Lean calls. The score estimates search promise, not theorem truth.

The state priority score combines signals that estimate whether expanding a state is likely to finish an active proof:

$$\begin{aligned}\mathrm{score}(s) = {} & \alpha_{\mathrm{in}}\mathrm{score}_{\mathrm{incoming}}(s) + \alpha_{\mathrm{last}}\mathrm{score}_{\mathrm{last}}(s) + \alpha_{\mathrm{prog}}\mathrm{bonus}_{\mathrm{committed}}(s) \\ & - \alpha_{\mathrm{depth}}\mathrm{depth}(s) - \alpha_{\mathrm{retry}}\mathrm{tacticTries}(s) - \alpha_{\mathrm{skel}}\mathrm{skeletonTries}(s) - \alpha_{\mathrm{bad}}\mathrm{badSkeletonRounds}(s).\end{aligned}$$

Incoming skeleton score transfers priority from a promising decomposition to its children. The committed-skeleton progress bonus favors children of the active skeleton, especially the last open child. Depth and retry penalties reduce priority for states that are deep, repeatedly attempted, or attached to skeletons with little progress.

Skeletons are scored before commitment or reservation:

$$\mathrm{score}(a_{\mathrm{skel}}) = \beta_{\mathrm{env}} r_{env} + \beta_{\mathrm{parent}}\mathrm{score}_{\mathrm{parent}} - \beta_{\mathrm{depth}}\mathrm{depth}(\mathrm{parent}) - \beta_{\mathrm{repair}}\mathbb{1}_{\mathrm{repaired}}.$$

This score prefers skeletons that are clean, recoverable, and proposed from promising parent states. A repaired skeleton can still be useful, but a clean skeleton is preferred when other signals are similar.

In practice, the score must interact with the search budget. A skeleton that is only slightly worse than the best one may still be worth reserving if the parent state is important and the remaining queue is small. Conversely, a very poor skeleton should not remain in the queue just because it is technically valid. The scoring policy is therefore not a universal ranking of mathematical quality; it is a local decision rule for allocating limited verification effort.

The formula is intentionally heuristic rather than normative. Its purpose is to prefer states that are close to completing the currently committed proof structure, not necessarily states that are globally shortest or most elegant. A deep state with one nearly solved child may outrank a shallow state with no clear proof direction if the committed skeleton is near completion. Conversely, a deep state with repeated failed local proof attempts and stale skeletons should drift downward in the queue even if it has not yet exceeded its hard search limits. This is the practical difference between best-first search and a depth-ordered rollout: the former tries to spend effort where it is most likely to close something useful, while the latter merely walks level by level.

Queue pruning is equally important. A proof search graph can accumulate many open states that are no longer promising because they are dominated by a better sibling, belong to a stale skeleton, or have already consumed too many attempts. If the queue grows without pruning, the system wastes verification time on states that are unlikely to contribute to the final proof. GammaZero therefore treats pruning as part of the search policy, not as an afterthought. The pruning count is one of the recorded metrics because it measures how aggressively the search focused on promising open states.

Skeleton scoring plays a similar role at the decomposition level. A skeleton that creates a compact and recoverable parent proof may be activated immediately, while a skeleton that repeatedly repairs into a fragile structure may remain reserved or be discarded. The score does not claim to predict theorem truth; it only predicts whether the candidate is likely to be useful inside the current proof graph. This separation between utility and truth is what allows GammaZero to keep using partially successful actions as data even when the final theorem is not solved.

6.10 Final Proof Construction

When child states are solved, their proof bodies are stitched into the corresponding named child subgoals in the parent skeleton. This process propagates upward: solved children close a skeleton, a solved skeleton closes its parent state, and the root theorem is reconstructed from the selected action chain.

After stitching, dependency analysis may identify declarations that are solved but unused by the final proof expression. Such unused declarations may be removed, and Lean is asked to verify the cleaned theorem again. The accepted final proof must contain no real `sorry` or `admit`; placeholder-compilable scripts are retained only as scored training records.

This final verification step deserves emphasis because it closes the loop between search and proof correctness. The search graph may contain many branches, repaired candidates, reserved skeletons, and unused intermediate declarations. None of those artifacts should leak into the accepted theorem. The final checker therefore acts on the reconstructed proof body, not on an abstract summary of the search. If the cleaned proof no longer verifies, the theorem is not considered solved. In other words, the search record may be richer than the final proof, but the final proof remains the only object that matters for correctness.

7 Dense Reward and Credit Assignment

7.1 Immediate Reward

For an action a generated at state s, let $L_{\text{orig}}(a)$ be the list of clean proof lines in the original generated action, excluding blank lines and comment-only lines. Let $L_{\text{patch}}(a)$ be the clean proof lines after Sorrifier recovery. The number of surviving lines N_{surv} is computed by sequence matching between $L_{\text{orig}}(a)$ and

$L_{\text{patch}}(a)$. A line is counted as surviving only if it appears unchanged and in the same relative order in the patched proof.

Let $N_{\text{orig}} = |L_{\text{orig}}(a)|$, and let N_{dead} be the number of Lean diagnostics corresponding to unused or ineffective code, such as warnings that a declaration is unused or that a tactic does nothing. The immediate environment reward is:

$$r_{env}(s,a) = \begin{cases} \dfrac{\max(0, N_{\text{surv}} - N_{\text{dead}})}{N_{\text{orig}}}, & N_{\text{orig}} > 0, \\ 0, & N_{\text{orig}} = 0. \end{cases}$$

This reward is a recoverability signal rather than a proof certificate. It rewards preservation of the model's original proof structure after recovery, while penalizing deleted, patched, unused, or ineffective code. Final proof success is still determined only by Lean verification without any `sorry` placeholder.

7.2 Dependency-aware Structural Reward

For skeleton actions, syntactic validity alone is insufficient. A skeleton may introduce intermediate claims that are syntactically valid but irrelevant, or may leave placeholder-containing declarations that make the resulting proof incomplete. To evaluate whether generated subgoals actually contribute to the parent proof, GammaZero performs expression-tree dependency analysis on the elaborated Lean proof.

After search, the system stitches solved child proof fragments back into the parent skeleton and asks Lean to elaborate the resulting proof script. The script may be fully verified or only placeholder-compilable due to remaining `sorry` terms. A separate expression-dumping process exports the Lean 4 expression tree into JSON. The dependency analyzer traverses this tree, including `bvar`, `fvar`, `app`, `lam`, `forallE`, and `letE` nodes, to determine whether each generated declaration is used by the parent proof expression.

The traversal is De Bruijn-aware. When the analyzer enters a binder such as `lam`, `forallE`, or `letE`, it shifts the target De Bruijn index so that references to bound variables are attributed to the correct local declaration. The analyzer also detects `sorryAx` in the expression tree. Combining usage detection with placeholder detection gives four structural cases:

- **Solved and used:** the generated declaration has a solved proof body and is used by the elaborated parent proof.
- **Unresolved and used:** the generated declaration still contains a placeholder and is used by the parent proof, so the parent is not a complete proof.
- **Solved but unused:** the generated declaration is complete but redundant for the parent proof.
- **Unresolved and unused:** the generated declaration is an unused unresolved obligation.

Let n_{core} be the number of solved-and-used declarations, n_{benign} the number of solved-but-unused declarations, and $n_{\text{unused\_failed}}$ the number of unresolved-and-unused declarations. The dependency reward used by the implementation is:

$$r_{dep}(s,a) = \frac{n_{\text{core}}}{n_{\text{core}} + 0.5\, n_{\text{benign}} + 2.0\, n_{\text{unused\_failed}}}.$$

If the denominator is zero, the reward is defined as 0. This reward favors skeletons whose generated subgoals are both solved and actually used, mildly penalizes solved but unused work, and strongly penalizes unused unresolved declarations. Unresolved-and-used declarations are handled as structural failures because the stitched proof still depends on a placeholder-containing declaration.

The formula can be read as a normalized usefulness ratio. The numerator counts declarations that both survive and matter. The denominator counts those declarations plus discounted forms of waste. A

solved-but-unused declaration is not as harmful as an unresolved unused obligation because it does not block final verification, but it still consumes search effort and may indicate that the skeleton created unnecessary structure. An unresolved unused obligation is worse because it both consumes effort and leaves unfinished code in a placeholder-compilable script. This is why the implementation weights it more heavily.

For example, suppose a skeleton creates three named subgoals. Two are solved and used by the parent proof, while one is solved but unused. Then $n_{\text{core}} = 2$, $n_{\text{benign}} = 1$, and $n_{\text{unused\_failed}} = 0$, giving

$$r_{dep} = \frac{2}{2 + 0.5} = 0.8.$$

The skeleton receives substantial structural credit because most of its work contributes to the proof. If the third subgoal were unresolved and unused instead, the reward would become

$$r_{dep} = \frac{2}{2 + 2.0} = 0.5.$$

This lower value reflects the fact that the skeleton created unfinished work that did not help the final assembly. If an unresolved declaration is used by the parent proof, the issue is more serious: the parent depends on a placeholder-containing declaration, so the skeleton cannot be treated as structurally solved even if some of its other children are useful.

In addition to expression-tree usage analysis, the implementation performs greedy pruning for suspected redundant intermediates. It attempts to remove candidate unused declarations and re-runs Lean verification. If the proof remains complete, the removed declaration is confirmed as redundant. This step helps distinguish genuinely used proof dependencies from artifacts that merely appear in the surface script.

The De Bruijn-aware traversal is necessary because Lean's elaborated expressions do not always preserve surface-level names in the way a human reader expects. Under binders, variables are represented by indices rather than by the printed names that appeared in the script. If the analyzer failed to shift target indices when entering a binder, it could attribute a reference to the wrong local declaration. Such an error would be especially damaging for skeletons, because dependency reward is meant to distinguish a useful intermediate claim from a redundant one. The analyzer therefore works over the elaborated expression tree rather than over raw text.

This dependency analysis is an attribution method for the proof that was actually found. It does not claim that an unused declaration is mathematically irrelevant in every possible proof, only that the selected stitched proof did not use it. This limitation is acceptable for reward generation because the trajectory is tied to one concrete proof attempt. A future policy update should learn from the behavior of the attempted proof graph, not from hypothetical alternative proofs that were never constructed.

7.3 AND/OR Backup

The framework assigns action values differently for local proof actions and skeleton actions.

For local proof actions, the action is expected to close the current proof state directly. Its value is:

$$Q(s, a) = r_{env}(s, a) + W_{solve} \cdot \mathbb{1}_{\text{solved}}(s, a),$$

where $\mathbb{1}_{\text{solved}}(s, a) = 1$ only if Lean reports that the current goal is closed without `sorry` or `admit`. Otherwise, the action receives only its immediate environment reward.

For OR nodes, corresponding to proof states, the value is determined by the best available action:

$$V(s) = \max_{a \in \mathcal{A}(s)} Q(s, a).$$

For skeleton actions, the value follows hard AND semantics. A skeleton introduces child proof states $S' = \{s'_1, \ldots, s'_k\}$, all of which must be solved for the decomposition to succeed. The action value is:

$$Q(s,a) = r_{env}(s,a) + \mathbb{1}_{\text{solved}}(s,a)\left(r_{dep}(s,a) + \gamma \min_{s' \in S'} V(s')\right).$$

Here, $\mathbb{1}_{\text{solved}}(s,a) = 1$ only when all child proof states generated by the skeleton are solved. If even one child remains open when the search limits are reached, the skeleton is not considered solved and receives only:

$$Q(s,a) = r_{env}(s,a).$$

The min operator implements the AND semantics of proof decomposition: the value of a skeleton is bottlenecked by its weakest child branch. This prevents the model from receiving delayed structural credit for a decomposition that solves only easy subgoals while leaving an essential obligation unresolved. Failed or partial actions keep their immediate rewards and diagnostics for analysis, but they do not solve the parent proof state and they do not receive delayed structural credit.

In contrast, the max operator at OR nodes reflects ordinary action choice: a proof state is considered valuable if at least one outgoing action leads to a strong proof continuation. Thus, the framework uses max backup over alternative actions and min backup over conjunctive skeleton children.

This backup rule also explains why local proof and skeleton actions should not be mixed naively during optimization. A local proof action receives a direct solve bonus when it closes the current state. A skeleton may receive no delayed value until its weakest child is solved, even if the skeleton itself was a good decomposition. If both action types were optimized with a single undifferentiated advantage estimate, the model could be biased toward short local proofs simply because their rewards are observed earlier. Keeping the action types explicit makes it possible to compare local proof actions with local proof actions and skeleton actions with skeleton actions, or to design future objectives that normalize across action families.

7.4 Reward Computation and Training Data Generation

The implemented system produces scored proof-search trajectories rather than an end-to-end optimized policy checkpoint. Each sampled action is executed, optionally recovered, inserted into the AND/OR graph, and assigned immediate and delayed rewards according to the formulations above. The resulting records contain the serialized proof state, action type, generated Lean code, recovery result, child proof states, graph status, priority-queue metadata, skeleton commitment metadata, and final action value.

These records can be used as training data for later policy optimization. Local proof actions and skeleton actions should be grouped separately because their value functions are structurally different: local proof actions receive direct solve bonuses, whereas skeletons receive delayed credit only when all generated child subgoals are solved. Therefore, comparing local proof actions and skeleton actions in the same optimization group would mix incompatible reward scales.

7.5 Trajectory Export and Final Verification

For a successful root theorem, GammaZero reconstructs the proof by recursively inserting solved child proof bodies into the named subgoals of the committed parent skeletons. Reserved skeletons and failed actions remain in the search record, but they are not part of the selected final proof. After insertion, dependency analysis may remove solved declarations that are unused by the elaborated proof expression. The resulting theorem is submitted to Lean again. The theorem is accepted as solved only if this final verification succeeds without real `sorry` or `admit`.

Due to computational resource constraints, this thesis does not train a new policy model with GRPO or another reinforcement learning algorithm. Instead, the contribution is the search-and-reward framework that generates dense, structured feedback from Lean executions. Policy optimization using the collected trajectory data is left for future work.

8 Experiments

8.1 Experimental Scope

The goal of the experiments is to evaluate the implemented hierarchical search and reward-assignment framework, rather than to train a new policy model end-to-end. Due to computational resource constraints, this thesis does not perform full GRPO training. Instead, the evaluation focuses on whether the proposed system can generate structured proof-search trajectories, recover invalid Lean generations, expand skeleton-created subgoals, and compute meaningful immediate and structural rewards.

The evaluation questions are intentionally operational rather than purely descriptive. Does best-first selection concentrate work on a smaller number of promising states than a level-wise schedule would? Do the local-proof-first and skeleton-commitment rules reduce unnecessary decomposition? Does scaffold-aware subgoal extraction preserve the exact target hole while leaving sibling subgoals unchanged? Does the dependency reward distinguish useful skeletons from cluttered ones? These questions matter because the thesis is about building a search-and-data pipeline, not only about solving a benchmark in isolation.

The experiments therefore measure the behavior of the following implemented components:

- best-first priority-queue search over an AND/OR proof graph;
- local-proof-first expansion and skeleton commitment with reserved fallback states;
- execution of local proof and skeleton actions in Lean 4;
- AST-guided recovery of invalid generations through the Sorrifier;
- extraction of scaffold-aware subgoals from `sorry` placeholders;
- line-based immediate reward computation;
- dependency-aware structural reward computation from elaborated Lean expressions;
- queue pruning and final proof stitching.

Each metric is tied to a specific stage of the pipeline. Root solved rate without placeholders is the final correctness metric. Generated actions and Lean verification calls measure search effort. Repaired candidates and subgoal extraction failures measure how often the system had to recover from model errors. Inserted skeletons and commitment/fallback counts measure the decomposition policy. Queue pruning counts measure how aggressively the search filtered open states. Average solved depth measures how far the system had to descend before a proof was completed. Dependency-class distributions show how often skeletons produced useful versus wasteful subgoals. Together, these metrics describe both search quality and data quality.

The resulting search records can be viewed as scored proof-search trajectories. They are suitable for future policy optimization, but this thesis reports only the search, recovery, and reward-computation results obtained from the current implementation.

In addition to aggregate metrics, the experiments should also inspect representative trajectories. A useful trajectory is not merely one that solves the root theorem; it is one that reveals where a tactic closed a state, where a skeleton introduced a useful subgoal, where repair preserved a large fraction of the generated code, and where dependency analysis correctly identified a solved but unused declaration. These qualitative

checks are especially important because the framework is designed to produce training data for later models. A trajectory that looks mathematically promising but is poorly structured for credit assignment is less useful than a slightly shorter trajectory that cleanly separates solved, redundant, and unresolved subgoals.

8.2 Setup

Lean environment. All experiments are conducted in Lean 4 using version `v4.29.0-rc1`. The project uses Mathlib (The mathlib Community, 2020) from the `leanprover-community/mathlib4` repository, pinned to revision:

`33a7291a345d718bca41f8ae8a9bae365d8f2a3e`

The Mathlib input revision is `v4.29.0-rc1`. This pinned environment ensures that all Lean executions, diagnostics, and proof-state reports are evaluated under a fixed library version.

Hardware. Experiments are run on a Linux workstation with an NVIDIA RTX 3090 GPU with 24GB VRAM, 64GB system RAM, and an Intel Xeon Silver 4114 CPU. The CPU has 20 logical threads, with 10 physical cores and 2 threads per core. Lean execution is parallelized using multiple Lean workers.

Policy model. Candidate local proof actions and skeleton actions are generated using `Gemini 3 Flash Preview`. The model is used as a fixed proposal mechanism for search-data generation; no additional fine-tuning or reinforcement learning update is performed in this thesis. The model samples Lean code conditioned on serialized proof states and task-specific prompts for local proof or skeleton action generation.

Search configuration. For each selected proof state, the search samples candidate actions subject to global and per-state generation limits. Tactic attempts are tried before skeleton requests, and the priority queue controls which open state is expanded next. The maximum search depth is set to $D = 10$, and the maximum number of graph nodes per theorem is set to $N = 512$.

Lean execution uses 4 parallel workers. Each Lean call has a timeout of 120 seconds. The Sorrifier is allowed up to 100 recovery cycles for a failed generation.

Generation configuration. The maximum generation length is set to 8192 new tokens. Sampling uses temperature 1.0, with the default nucleus-sampling setting of the backend API, approximately $p = 0.95$ when applicable. The generated output may include a `<think>` block, but only the extracted Lean code block is submitted to Lean for execution.

8.3 Dataset

The evaluation uses the validation split of miniF2F-v2, introduced by Ospanov et al. (2025b) as a corrected version of the original miniF2F benchmark (Zheng et al., 2022). The original miniF2F benchmark contains 488 formalized olympiad-level mathematics problems and has become a standard benchmark for neural theorem proving. However, the revisited study identifies mismatches and errors between informal and formal statements in the original benchmark and provides corrected versions with verified alignment between the informal problem statements and the corresponding Lean formalizations.

In this thesis, we evaluate on the `miniF2F-v2s` validation split, the simplified aligned version of miniF2F-v2. The dataset is suitable for testing both local tactic generation and higher-level proof decomposition because the problems cover competition-style mathematics such as algebra, number theory, and inequalities.

8.4 Metrics

We report root solved rate without placeholders, pass@k, generated actions, Lean verification calls, repaired candidates, inserted skeletons, commitment/fallback count, queue pruning count, average solved depth, policy-check rejection rate, subgoal extraction failure rate, and dependency-class distribution.

Root solved rate is computed only from final Lean-verified proofs that contain no real `sorry` or `admit`. This metric is deliberately strict: placeholder-compilable proofs, partially solved skeletons, and repaired scripts do not count as solved roots. Pass@k is used when multiple independent search runs or sampled attempts are available for the same theorem; it estimates whether at least one attempt within the allowed set solves the theorem.

Generated actions count all local proof and skeleton proposals that pass through the model interface. Lean verification calls count executions sent to the persistent Lean workers, including checks used during repair and final verification. These two counts are separated because one generated action may trigger several Lean calls if it requires repair, while a rejected action may trigger no full verification if it fails a policy check first. Reporting both numbers makes the compute profile clearer.

Repaired candidates measure how often the Sorrifier was needed. A high repair rate can be read in two ways: the model may be producing many invalid candidates, but the recovery module may also be successfully extracting signal that would otherwise be lost. For this reason, repair rate should be interpreted together with line-survival reward and downstream dependency reward. A repaired candidate that preserves useful structure is different from one that collapses to a single placeholder.

Inserted skeletons, commitment counts, and fallback counts measure whether decomposition is being used in a controlled way. If skeleton generation is frequent but commitment is rare, the model may be proposing invalid or low-quality decompositions. If fallback is frequent, the initial skeleton ranking may be weak or the search limits may be too tight. If commitment is common but child solve rates are low, the decomposition may expose subgoals that are too hard for the current local proof policy.

Dependency-class distribution is the main structural metric for skeleton quality. A good skeleton should increase solved-and-used declarations, keep solved-but-unused declarations low, and avoid unresolved unused subgoals. The distribution also helps diagnose reward hacking. If many skeletons create large numbers of unused declarations while still receiving high immediate reward, then syntactic survival is not enough and the dependency term is necessary.

8.5 Ablations

We evaluate the following ablations: no Sorrifier, no scaffold-aware subgoal solving, no local-proof-first rule, no skeleton commitment, no dependency reward, FIFO/level-wise selection versus priority queue, and different generated-action and depth limits.

Each ablation isolates one design choice. Removing the Sorrifier tests whether partial-credit signals are actually helping or merely adding noise. Removing scaffold-aware subgoal solving tests whether exact parent-scaffold tracking matters beyond ordinary placeholder extraction. Removing the local-proof-first rule tests whether the search benefits from trying cheap local closure before decomposing a goal. Removing skeleton commitment tests whether multiple incompatible decompositions cause the search to diffuse its effort. Removing dependency reward tests whether structural usefulness matters once syntactic survival has already been counted. Comparing FIFO or level-wise selection against a priority queue tests whether best-first ordering really matters for these proof graphs.

The experiments are not meant to produce a single victory number. They should show a profile of how the system behaves under different resource budgets and search policies. That profile is useful even if root solve rate remains modest, because the thesis contribution is the generation of scored trajectories and search structure. In other words, a partially successful system can still be a good thesis result if it makes

proof search more analyzable, more recoverable, and better suited to future optimization.

8.6 Analysis Protocol

The analysis proceeds in three layers. The first layer is theorem-level correctness: how many roots are solved, how many attempts are needed, and how often final stitching succeeds. The second layer is search behavior: how many states are opened, how the priority queue allocates expansions, and how often states are pruned. The third layer is data quality: how much repair signal is recovered, how skeleton dependencies are classified, and how many actions are suitable for future optimization.

This layered analysis prevents misleading conclusions. A configuration might solve slightly fewer roots but produce cleaner trajectories with fewer unused subgoals. Such a configuration may be valuable for training-data generation. Conversely, a configuration might solve a theorem by exploring a large number of low-quality branches, making it less attractive if the goal is to create efficient search records. The experiments should therefore report both proof success and the structure of the path that led to success.

When comparing ablations, the same theorem set, model, generation parameters, Lean version, timeout, and maximum search limits should be used. This keeps differences attributable to the disabled component rather than to sampling or environment changes. Because LLM sampling is stochastic, results should also record random seeds or run identifiers where possible. If multiple runs are too expensive, the thesis should still report enough metadata for the experiment to be reproduced.

8.7 Reproducibility Considerations

Reproducibility is difficult in LLM-guided theorem proving because several components can change independently. The model backend may update, sampling can be stochastic, Lean and Mathlib versions can change theorem names or elaboration behavior, and worker timeouts can depend on hardware load. For this reason, the experimental setup records the Lean version, Mathlib revision, model name, generation parameters, worker count, timeout, search limits, and dataset split. These details are not administrative noise; they define the environment in which a proof attempt is meaningful.

The most important reproducibility boundary is the Lean environment. A proof that verifies under one Mathlib revision may fail under another because a lemma was renamed, a simp rule changed, or an instance search path became different. The thesis therefore treats the pinned Lean and Mathlib versions as part of the experimental artifact. Search records should be interpreted relative to that environment, and future comparisons should either use the same environment or explicitly report the migration.

LLM sampling introduces a second source of variance. Even with fixed prompts and temperature, hosted model APIs may not provide perfect determinism. The thesis therefore emphasizes aggregate search metadata and trajectory structure in addition to root solve rate. If one run solves a theorem and another does not, the difference may come from sampling rather than from the architecture. Recording generated actions, priority scores, repair results, and dependency classes makes it possible to inspect why a run succeeded or failed.

8.8 Threats to Validity

There are four main threats to validity. The first is benchmark validity. miniF2F-style datasets are useful because they provide formal theorem statements, but they do not cover all styles of mathematics or all Lean proof patterns. A system that works on olympiad algebra may behave differently on topology, category theory, or large-library engineering proofs. This thesis therefore treats miniF2F-v2s as a controlled evaluation setting rather than as a complete measure of theorem-proving ability.

The second threat is model dependence. GammaZero is designed to be model-agnostic, but the quality of generated local proof actions and skeleton actions strongly affects the observed search behavior. A stronger model might need fewer repairs and produce better skeletons; a weaker model might make the

search appear worse than the architecture itself. The experiments should therefore report the proposal model clearly and avoid claiming that results transfer automatically to all LLMs.

The third threat is heuristic dependence. Priority scores, skeleton commitment thresholds, and stale skeleton rules are hand-designed in the current implementation. They are intended to be reasonable engineering choices, not optimal search theory. Ablations partly address this threat by comparing priority-queue search against simpler schedules and by disabling individual components. Still, the exact numbers should be interpreted as evidence about this implementation rather than universal constants.

The fourth threat is reward interpretation. Dense reward can make partial progress visible, but it can also overemphasize syntactic survival if not paired with structural checks. GammaZero mitigates this by combining line-based recovery reward with dependency analysis and final Lean verification. Nevertheless, the reward values should be understood as training signals, not as mathematical measures of elegance or insight.

9 Conclusion and Limitations

This thesis presents a hierarchical framework for Lean 4 theorem proving that separates local proof resolution from skeleton decomposition. The design addresses sparse reward through AST-guided recovery and handles decomposition credit through dependency-aware AND/OR backup. The key correctness boundary is that patched scripts with `sorry` are used only for learning signal; final success requires a fully verified Lean proof without placeholders.

The current design still has limitations. Dependency-based credit assignment reflects the proof that was found, not necessarily the shortest or mathematically minimal proof. Open subgoals within finite search limits cannot always be distinguished from genuinely impossible subgoals. The effectiveness of skeleton generation depends on the policy model's ability to propose useful intermediate claims. These limitations should be evaluated empirically in the experiments.

There are also broader limitations that are easier to state than to eliminate. The model interface assumes that the LLM can produce output that is at least parseable into Lean code blocks. If the model drifts into natural language, malformed markdown, or repetitive syntax errors, the Sorrifier can recover only a fraction of the damage. The scaffold-aware contract assumes that subgoals can be cleanly associated with a single target hole, which is true for the proof bodies GammaZero is designed to handle but may be awkward for very irregular generated code. The priority queue assumes that the heuristic score is at least informative enough to rank open states roughly by promise; if that score is poor, the system can still search, but it will search more expensively.

The dependency reward also has an important interpretive limit. It measures usage in the elaborated proof that was found, not a global theorem-theoretic notion of necessity. A declaration can be unused in the final stitched proof and still have been useful as an intermediate exploration step. Likewise, an unresolved child may block the final theorem even if it looks locally reasonable. These are not bugs in the reward definition; they are reminders that the thesis is about search data, not formal proof minimization.

Future work naturally follows from this framing. The collected search records could be used for policy fine-tuning or GRPO-style optimization. Retrieval could be integrated so that skeleton prompts and local proof prompts receive relevant premises automatically. Larger or more diverse theorem libraries could be added so that the search graph contains a broader variety of decomposition patterns. The skeleton scorer could also be replaced by a learned ranking model once enough trajectories have been collected. Each of these directions would build on the current thesis, but none of them is required for the present contribution: a verifiable, scaffold-aware, priority-queue proof-search system that produces structured reward data.

Another direction is to improve the model interface itself. The current separation between local proof and skeleton prompts is simple and inspectable, but a future system could condition the prompt on search

history, failed local proof attempts, dependency feedback, or retrieved examples of similar proof patterns. A model might learn that certain goals should be attacked directly while others should be decomposed. It might also learn to propose smaller skeletons that introduce only subgoals likely to be solved by the current local proof policy. Such improvements would turn the current hand-designed local-proof-first rule into a learned decision about when decomposition is useful.

The final long-term goal is a closed loop: generate search trajectories, score them with Lean-backed signals, train a better proposal model, and then use the improved model to generate higher-quality trajectories. This thesis implements the first half of that loop. It defines the graph structure, verification contracts, repair mechanism, dependency analysis, and trajectory export needed for such training. The remaining optimization step is left for future work because it requires substantial compute and careful experimental design. Keeping that boundary explicit avoids overstating the contribution while still showing how the system can support future reinforcement learning.

The thesis also leaves room for better proof-object analysis. The current dependency analyzer focuses on whether generated declarations are used by the elaborated proof expression and whether they contain placeholders. A richer analyzer could measure proof-term size, count references to library lemmas, detect whether a declaration is used only through a trivial wrapper, or compare multiple stitched proofs for the same theorem. Such analysis would help distinguish merely valid proofs from compact or pedagogically useful proofs. That is outside the current scope, but the graph representation makes the extension natural.

Finally, the system could benefit from stronger feedback to the model at generation time. At present, the model is prompted with the current goal and action type. Future prompts could include summaries of failed attempts, names of reserved skeletons, dependency feedback from previous decompositions, or retrieved examples from similar theorems. This would move GammaZero from a passive sampler toward an interactive prover that explains to the model what has already been tried. The strict verification boundary would remain unchanged: whatever the model proposes, Lean must still check the final proof.

References

Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics, 2023.

Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. HOList: An environment for machine learning of higher order logic theorem proving. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 454–463. PMLR, 2019. URL https://arxiv.org/abs/1904.03241.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-Prover: Deep and broad reasoning for automated theorem proving, 2025.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing. doi: 10.1007/978-3-030-79876-5_37.

Kefan Dong, Arvind Mahankali, and Tengyu Ma. Formal theorem proving by rewarding LLMs to decompose proofs hierarchically, 2024.

Albert Q. Jiang, Wenda Li, Szymon Tworkowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Mi loś,

Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers, 2022a.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2022b.

Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Mirek Olsák. Reinforcement learning of theorem proving, 2018.

Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving, 2022.

Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. APOLLO: Automated LLM and Lean collaboration for advanced formal reasoning, 2025a.

Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. miniF2F-Lean revisited: Reviewing limitations and charting a path forward, 2025b.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3372885.3373824.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models, 2022.

Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.15626.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=9ZPegFuFTFv.

A Running Example

Consider the following simple theorem:

```
theorem split_imp
(P Q R : Prop)
(hpq : P -> Q)
(hpr : P -> R) :
P -> Q /\ R := by
```

The initial goal is to prove $P \to Q \wedge R$. A local proof action may solve the goal directly:

```
intro hp
exact And.intro (hpq hp) (hpr hp)
```

If Lean accepts this proof body and no goal remains, the current proof state is solved. This action does not create child proof states. It is a complete local attempt for the current target hole.

A skeleton action may instead expose the two intermediate facts Q and R:

```
intro hp
have hq : Q := by
  sorry
have hr : R := by
  sorry
exact And.intro hq hr
```

This skeleton does more than list two subgoals. It also gives the proof structure that will use the subgoals after they are solved. The declarations `hq` and `hr` are the intermediate facts, and the final line

```
exact And.intro hq hr
```

explains how they are combined to prove the parent goal.

The two placeholders create two child subgoals. When the system focuses on the first child, the scaffold keeps the surrounding proof structure, but only the proof of `hq` remains as the target:

```
intro hp
have hq : Q := by
  sorry
have hr : R := by
  admit
exact And.intro hq hr
```

The current node is responsible only for replacing the target `sorry`. The sibling subgoal `hr` is temporarily represented by `admit`, meaning that it is not solved by this node. In this scaffold, the proof of `hq` can be:

```
exact hpq hp
```

When the system focuses on the second child, the roles are reversed:

```
intro hp
have hq : Q := by
  admit
have hr : R := by
  sorry
exact And.intro hq hr
```

Now the target is the proof of `hr`, and the proof body can be:

```
exact hpr hp
```

The use of `admit` here is temporary. It allows the system to check one child proof inside the parent scaffold while the sibling child is still open. These checks are not final proofs of the theorem. They only verify whether a proposed replacement is valid in the proof context where it will later be used.

After both children are solved, GammaZero inserts the solved proof bodies back into the original skeleton:

```
intro hp
have hq : Q := by
  exact hpq hp
have hr : R := by
  exact hpr hp
exact And.intro hq hr
```

The completed scaffold is then checked again by Lean. This final check is essential: the parent proof is accepted only if the stitched proof contains no remaining `sorry` or `admit` and Lean verifies the whole block.

This example illustrates the boundary between the two action types. A local proof action tries to close the current target directly. A skeleton action introduces intermediate proof structure and may create child subgoals. Each child is solved in the scaffold where it was created, and the parent skeleton succeeds only when all required children are solved and the completed proof is accepted by Lean.

The example is simple, but the same pattern applies to larger proofs. A difficult theorem may require intermediate bounds, normalization steps, case-specific facts, or auxiliary lemmas that are not explicit in the theorem statement. A skeleton makes such hidden structure explicit. Instead of forcing the search to discover a long tactic sequence step by step, the system can first propose a proof structure, then solve the child subgoals inside that structure.

B Sorrifier Examples and Case Studies

This subsection illustrates how the Sorrifier converts different Lean failures into placeholder-compilable scripts and how the line-based survival reward is computed. In all examples, blank lines and comments are excluded from the line count.

Case 1: Parser error. Consider a generated proof body containing a malformed tactic:

```
have h := 1
apply h (
```

Lean reports a parser error because the opening parenthesis is not closed. The Sorrifier localizes the failing tactic line and replaces it with `sorry`:

```
have h := 1
sorry
```

The original body contains two clean lines. One line survives unchanged, while the malformed tactic is replaced. Therefore, the immediate reward is:

$$r_{\text{env}} = \frac{1}{2} = 0.5.$$

Case 2: Unresolved name and cascade cleanup. Consider a generated proof body that calls a nonexistent lemma:

```
apply nonexistent_lemma
rfl
```

Lean reports an unknown identifier error for `nonexistent_lemma`. After this failure, the following line is no longer useful in the recovered proof context. The Sorrifier therefore replaces the failed region with a single placeholder:

```
sorry
```

Although the original body has two clean lines, neither line survives unchanged in the patched body. Thus:

$$r_{\text{env}} = \frac{0}{2} = 0.0.$$

This example shows that recovery is not purely local at the textual level: an early failure may invalidate later commands that depend on the proof state created by the failed command.

Case 3: Skeleton partial recovery. The following skeleton introduces a valid intermediate fact before producing an invalid tactic:

```
have h : 1 + 1 = 2 := by
  rfl
apply nonexistent_lemma
exact h
```

The first two lines form a valid local proof of the intermediate fact. The third line fails because the lemma name cannot be resolved, and the final line becomes unusable after the failed application. The patched body is:

```
have h : 1 + 1 = 2 := by
  rfl
sorry
```

The original body has four clean lines. The first two lines survive unchanged, while the remaining two lines are replaced by a placeholder. Therefore:

$$r_{\text{env}} = \frac{2}{4} = 0.5.$$

This case illustrates why recovery is useful for skeleton actions: even when the generated proof does not fully close the goal, valid setup lines and intermediate claims can still be preserved and rewarded.

Case 4: Type mismatch. A minimal type mismatch example is:

```
exact 1
```

when the expected target is a proposition. Lean rejects the term because a numeral does not inhabit the expected proposition. The recovered body is:

```
sorry
```

No original clean line survives, giving:

$$r_{\text{env}} = 0.0.$$

C Reward Hacking Derivation

To prevent reward hacking through redundant valid nodes, benign redundant subgoals are assigned non-positive weights. The core concern is error dilution: a policy may try to hide invalid fragments by generating many valid but irrelevant nodes.

The implementation already encodes this bias directly in the dependency reward. Solved-and-used declarations receive the highest credit, solved-but-unused declarations are mildly discounted, and unresolved-and-unused declarations are strongly discounted. This makes redundant padding less attractive than honest structural progress without requiring a separate symbolic derivation.

D Design Assumptions and Limitations

A1. Syntactic validity after recovery provides useful local signal, but is not a proof of correctness.

A2. The Sorrifier preserves enough structure for partial-credit estimation.

A3. Dependency extraction is an attribution proxy for the specific proof found, not a guarantee of mathematical necessity.

A4. Separating local proof actions and skeleton actions is an appropriate abstraction for the target proof distribution.

A5. Local-proof-first action allocation is a heuristic that must be validated empirically.

A6. Search limits affect labels for unresolved subgoals; failure to solve within those limits should not be treated as a mathematical impossibility result.

A7. The training distribution must contain enough solvable states for delayed structural reward to be useful.

These assumptions are deliberately explicit because they define when the architecture is expected to be useful. If the model cannot produce parseable Lean at all, the Sorrifier can only record failures. If the theorem distribution rarely requires decomposition, skeleton actions add overhead without much benefit. If search limits are too small, many states will be labeled as unresolved within the current run even though they might be solvable with more attempts. The framework is therefore best understood as a tool for structured proof-search data generation under finite resources, not as a complete theorem prover that eliminates the need for a strong proposal model.

The assumptions also identify natural engineering tradeoffs. Increasing the generated-action limit gives the system more chances to find a proof but increases Lean verification cost. Allowing more skeletons per state increases decomposition diversity but risks spreading effort across incompatible proof plans. Making the Sorrifier more aggressive can recover more partial structure but may also blur the connection between the original generation and the recovered script. These tradeoffs are precisely why the experimental section records both proof success and search-process metrics.

E Extra Implementation Questions

The following points should be resolved before submission:

1. What exact Lean code format represents a skeleton?
2. Are local proof actions rejected whenever they leave any subgoal?
3. How are parser, elaboration, and type errors localized differently?
4. What is the semantic unit counted by r_{env}?
5. How is dependency usage extracted from Lean?
6. Is an unresolved subgoal labeled as an open state not solved within the current search limits?
7. Is the search truly best-first with priority-queue selection?
8. What concrete training objective is used?
9. Does final solved proof strictly forbid `sorry`? The answer should remain yes.